Infraestrutura de Software para Computação Gráfica

GBC204 – Computação Gráfica
Prof. Dr. rer. nat. Daniel Duarte Abdala

Infraestrutura de Software

- Para habilitar aplicações de CG, diversas camadas de software se fazem necessárias – Elas cobrem desde o interfaceamento direto com o hardware do pipeline gráfico até motores completos que facilitam o funcionamento de uma aplicação gráfica complexa;
- Basicamente, 5 camadas:

Camada 5:	Motores Gráficos
Camada 4:	Shaders e Compiladores de Shaders
Camada 3:	Utilidades Gráficas – Matemática, Imagens Fontes
Camada 2:	Bibliotecas de Suporte – Janela, Contexto, Entrada
Camada 1:	Abstração do Hardware – Drivers e APIs Gráficas



Camada 1: Abstração de Hardware

- **Problema chave**: como requisitar que o hardware desenhe algo?
- **Mais especificamente**: Diversidade de hardware!
- A tecnologia evoluiu muito nos últimos 30 anos, então consideremos um cenário mais atual: Programa que desenha um triângulo
  - Cada GPU tem registradores, pipeline e instruções diferentes;
  - Precisaria recompilar/reimplementar para cada hardware distinto;
  - Fabricantes mudam arquiteturas, Kepler ->Maxwell -> Pascal -> Turing -> Ampere (NVIDIA)

```
// HIPOTÉTICO - isso NÃO existe!
// Para NVIDIA RTX 4090:
write_register(NVIDIA_RTX4090, 0x3A4F, 0x0010);
write_register(NVIDIA_RTX4090, 0x3A50, vertex_data);

// Para AMD RX 7900 XTX (registradores completamente diferentes):
write_register(AMD_RDNA3, 0x8B2C, 0x0040);
write_register(AMD_RDNA3, 0x8B30, vertex_data);

// Para Intel Arc A770 (outra arquitetura):
write_register(INTEL_XE, 0x1F44, 0x0200);
```



Camada 1: Abstração de Hardware

- Solução:
  - Driver do Dispositivo (Driver Gráfico ou da placa de vídeo)
- O driver converte chamadas de alto nível em comandos específicos da GPU:
  - "Esconde" especificidades do hardware;
  - Cria um conjunto de chamadas de sistema homogêneo, indiferente do hardware instalado.

```
// O que sua aplicação escreve (OpenGL)
glUniformMatrix4fv(location, 1, GL_FALSE, matrix);
glDrawArrays(GL_TRIANGLES, 0, 3);

// O que o driver NVIDIA faz INTERNAMENTE (simplificado)
nvidia_push_command_to_ring_buffer(NVIDIA_COMMAND_UPDATE_CONSTANTS, matrix);
nvidia_update_pipeline_state(NVIDIA_PIPELINE_STATE_DRAW);
nvidia_submit_draw(3, NVIDIA_PRIMITIVE_TRIANGLES);
nvidia_ring_buffer_flush();

// O que o driver AMD faz INTERNAMENTE (diferente!)
amd_write_command_buffer(AMD_COMMAND_TYPE_CONSTANT_UPDATE, matrix);
amd_set_gfx_pipeline(AMD_GFX_PIPELINE_DEFAULT);
amd_dispatch_direct(AMD_PRIMITIVE_TRIANGLE_LIST, 3);
```



Camada 1: APIs

- O driver da placa de vídeo resolve o problema da homogeneização das chamadas do sistema, mas ainda é muito próximo do hardware.
- Programar Usando chamadas diretas as funções do Driver é muito próximo a programação Assembly da GPU (literalmente apenas um nível acima)
- Pouco Produtivo!
- **Solução**: Utilizar uma API que encapsule chamadas ao driver e implemente funções comumente utilizadas de maneira mais abstrata (ligeiramente mais abstratas e inteligíveis)
- Principais APIs:
  - OpenGL 1992
  - Vulkan 2016
  - Direct3D 12
  - Metal (Apple)



Camada 1: APIs - OpenGL

- API de **estado global** (máquina de estados)
- Abstração muito alta (esconde detalhes da GPU)
- Driver faz muito trabalho pesado
- Prob1: Overhead enorme (cada chamada glVertex é uma função)
- Prob2: Driver decide quando enviar para GPU (pode ser ineficiente)
- Prob3: Difícil para drivers otimizarem

```
// Seu código
glBegin(GL_TRIANGLES);
    glVertex3f(0, 1, 0);
    glVertex3f(-1, -1, 0);
    glVertex3f(1, -1, 0);
glEnd();

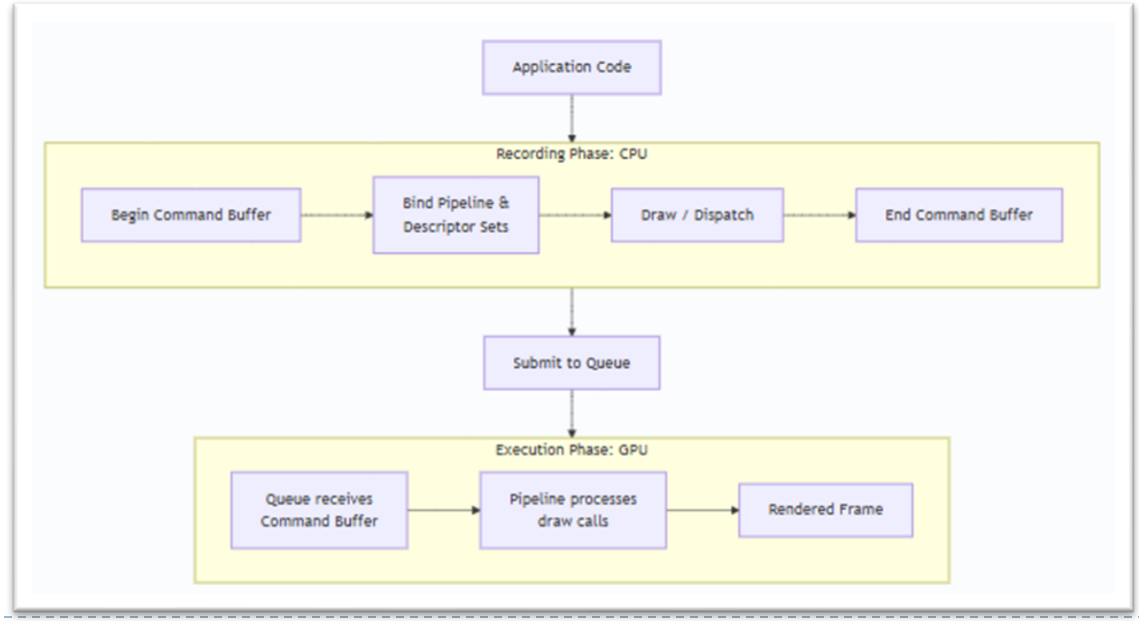
// Driver faz (simplificado):
// 1. Valida se estado atual permite desenho
// 2. Transforma vértices pelo modelo-visão-projeção
// 3. Aplica iluminação (se habilitada)
// 4. Envia vértices transformados para GPU
// 5. GPU rasteriza e pinta pixels
```



Camada 1: APIs - Vulkan

Filosofia: "Você Controla tudo, assuma a responsabilidade"

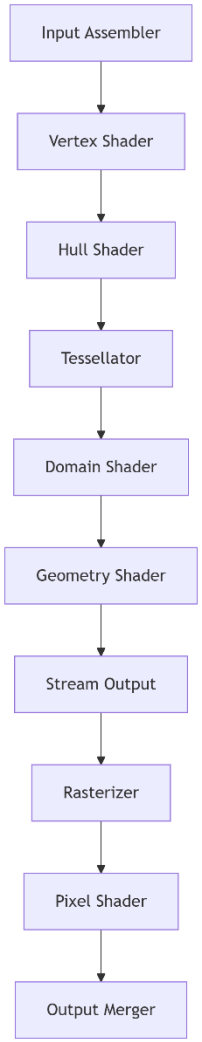
- O Vulkan foi anunciado oficialmente pela **Khronos Group** em 2015, durante a Game Developers Conference (GDC). Ele foi criado como a "iniciativa da próxima geração do OpenGL"
- Originalmente chamado de glNext com o objetivo de resolver as limitações do OpenGL principalmente em relação ao desempenho e à eficiência;





Camada 1: APIs - Direct3D 12 (Windows)

- A história do Direct3D começa em meados dos anos 90 e é marcada por aquisições estratégicas e evoluções técnicas radicais:
  - Origens (1995): A Microsoft comprou a empresa RenderMorphics, que havia desenvolvido uma API de gráficos 3D chamada "Reality Lab“
  - Era dos "Execute Buffers" (DirectX 3-5)
  - Renovação com DirectX 7 e 8
  - Era da Programabilidade (DirectX 9 ao 11)
  - Era de Baixo Nível - Atualidade (DirectX 12)





Camada 1: APIs – Metal (Apple)

- História:
  - Origens (2014): A Metal foi anunciada na WWDC de 2014 e lançada com o iOS 8 para dispositivos com chip Apple A7 (iPhone 5s) ou superior – Motivação: OpenGL não conseguia extrair todo o potencial dos chips Apple
  - Chegada ao Mac (2015): A Apple expandiu a Metal para o macOS com o OS X El Capitan
  - Metal 2 (2017): Aperfeiçoou a depuração, reduziu a carga da CPU e introduziu suporte a aprendizado de máquina (Machine Learning) e realidade virtual
  - Metal 3 (2022): Trouxe o MetalFX Upscaling, uma tecnologia que renderiza em resoluções mais baixas e usa inteligência artificial para dar o "upscale" com alta qualidade, similar ao DLSS da NVIDIA. Também adicionou suporte a Mesh Shaders
  - Metal 4 (2025): A versão mais recente foca em Neural Rendering



Camada 2: Bibliotecas de Suporte

- Resolve um problema fundamental:

"Como diabos eu abro uma janela e desenho algo nela sem usar um motor gráfico ou fazer chamadas diretas a camada 1?"

- **Problema Fundamental**: A GPU não sabe o que é uma janela!
- A GPU só sabe:
  - Desenhar pixels em um framebuffer (uma matriz de memória)
  - Executar shaders
  - Rasterizar triângulos
- **Então quem gerencia as janelas**: O Sistema Operacional



Camada 2: GLFW

- Graphics Library Framework
- A mais popular para OpenGL/Vulkan
  - Criação de janelas (Windows/Linux/macOS)
  - Contexto OpenGL ou Vulkan
  - Entrada de Dados (teclado, mouse, joystick)
  - Timers (para animações)
  - Open Source
  - https://www.glfw.org/

```c
int main() {
    // Inicializa GLFW
    glfwInit();

    // Configurações (opcional)
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    // Cria janela (SIMPLES!)
    GLFWwindow* window = glfwCreateWindow(800, 600, "Minha Janela", NULL, NULL);

    // Torna o contexto OpenGL atual
    glfwMakeContextCurrent(window);

    // Loop principal
    while (!glfwWindowShouldClose(window)) {
        // Seu código de renderização aqui

        glfwSwapBuffers(window);   // Troca os buffers
        glfwPollEvents();          // Processa eventos (teclado, mouse, fechar)

        // Verifica tecla ESC
        if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS) {
            glfwSetWindowShouldClose(window, GLFW_TRUE);
        }
    }

    glfwTerminate();
    return 0;
}
```



Camada 2: SDL – Simple DirectMedia Layer

- **Diferenças do GLFW**:
  - GLFW: foco em janela + contexto + entrada (para gráficos)
  - SDL: foco em games (janela + áudio + gamepad + timer + threading + eventos)

- https://www.libsdl.org/

```c
#include <SDL2/SDL.h>

int main() {
    SDL_Init(SDL_INIT_VIDEO);

    SDL_Window* window = SDL_CreateWindow("Janela SDL",
        SDL_WINDOWPOS_UNDEFINED, SDL_WINDOWPOS_UNDEFINED,
        800, 600, SDL_WINDOW_OPENGL);

    SDL_GLContext glContext = SDL_GL_CreateContext(window);

    SDL_Event event;
    int running = 1;

    while (running) {
        while (SDL_PollEvent(&event)) {
            if (event.type == SDL_QUIT) running = 0;
            if (event.type == SDL_KEYDOWN && event.key.keysym.sym == SDLK_ESCAPE)
                running = 0;
        }

        // Renderização aqui

        SDL_GL_SwapWindow(window);
    }

    SDL_GL_DeleteContext(glContext);
    SDL_DestroyWindow(window);
    SDL_Quit();

    return 0;
}
```



Camada 2: GLUT/FreeGlut (old+didático)

- OpenGL Utility Toolkit
- **Prós**: Extremamente simples (5 linhas para janela)
- **Contras**: Desatualizado, controle limitado, callbacks globais
- https://freeglut.sourceforge.net/

```c
#include <GL/freeglut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 600);
    glutCreateWindow("GLUT Window");
    glutDisplayFunc(display);
    glutMainLoop();  // NUNCA RETORNA!
    return 0;
}
```



Camada 2: Contexto Gráfico

- Ponte entre Janela e GPU
- O contexto é um **objeto do driver** que mantém todo o estado da sua aplicação gráfica.

Tabela Comparativa: GLFW vs SDL vs GLUT

Característica	GLFW	SDL	GLUT
Janela multiplataforma	✅	✅	✅
Contexto OpenGL	✅ (fácil)	✅ (manual)	✅ (automático)
Contexto Vulkan	✅	⚠️ (requer extensão)	❌
Áudio	❌	✅	✅ (básico)
Gamepad/Joystick	✅ (limitado)	✅ (completo)	✅ (básico)
Threading	❌	✅	❌
Clipboard	✅	✅	❌
Drag-and-drop	✅	✅	❌
Curva aprendizado	Baixa	Média	Muito Baixa
Manutenção	Ativo (2024)	Ativo (2024)	Morto (FreeGLUT mantido)
Uso típico	Ferramentas CG, engines leves	Jogos, aplicações multimídia	Prototipagem acadêmica



Camada 3: Utilidades Gráficas

- Essencial para qualquer aplicação gráfica séria
- Basicamente suporte a funções:
  - Matemática
  - Imagens
  - Fontes
- **Problema fundamental**: A GPU é boa para calcular muito rápido e de maneira massivamente paralela problemas em álgebra linear (multiplicação de matrizes)
- Mas...: deixa a desejar em basicamente todos os outros campos da matemática XD



Camada 3: Utilidades Gráficas - GLM

- GLM – OpenGL Mathematics
- GLM mimetiza a sintaxe do GLSL (shader language), facilitando o entendimento.

```
// Câmera LookAt (onde está, para onde olha, vetor "up")
glm::vec3 cameraPos(0.0f, 0.0f, 3.0f);
glm::vec3 cameraTarget(0.0f, 0.0f, 0.0f);
glm::vec3 worldUp(0.0f, 1.0f, 0.0f);

glm::mat4 view = glm::lookAt(cameraPos, cameraTarget, worldUp);

// Matriz de Projeção Perspectiva (campo de visão, aspect ratio, near, far)
glm::mat4 projection = glm::perspective(
    glm::radians(45.0f),  // FOV (graus convertidos para radianos)
    800.0f / 600.0f,      // Aspect ratio
    0.1f,                 // Near plane
    100.0f                // Far plane
);

// Matriz de Projeção Ortográfica (sem perspectiva)
glm::mat4 ortho = glm::ortho(-10.0f, 10.0f, -6.0f, 6.0f, 0.1f, 100.0f);
```

```
// SEM biblioteca (puro C++ - horrível)
float matrix[16] = {
    1,0,0,0,
    0,1,0,0,
    0,0,1,0,
    0,0,0,1
};  // Matriz identidade

// Multiplicar duas matrizes manualmente:
float result[16];
for(int i=0; i<4; i++) {
    for(int j=0; j<4; j++) {
        result[i*4+j] = 0;
        for(int k=0; k<4; k++) {
            result[i*4+j] += a[i*4+k] * b[k*4+j];
        }
    }
}
// Isso é PROPENSO A ERROS e devagar!

// COM GLM (claro e seguro):
glm::mat4 identity = glm::mat4(1.0f);
glm::mat4 result = a * b;  // SIMPLES!
```



Camada 3: Carregamento de Imagens

- **Problema**: Formatos de Imagens São Complexos
  - **JPEG**: Compressão com perda, codificação Huffman, DCT
  - **PNG**: Compressão sem perda (DEFLATE), canais alpha, pré-filtros
  - **BMP**: Simples, mas ineficiente
  - **TGA**: Suporta compressão RLE
  - **HDR**: Formato de ponto flutuante (iluminação)
- **Sem biblioteca**: Você teria que implementar decodificadores para cada formato
- Várias bibliotecas. Geralmente a mais usada é a STB, leve e simples



Camada 3: Renderização de Texto (FreeType)

- Porque texto é difícil em OpenGL/Vulkan?
  - Fontes TrueType são curvas Bezier (precisa rasterizar)
  - Cada glifo (caractere) tem tamanho diferente
  - Kerning (espaçamento entre pares como "AV")
  - Antialiasing (bordas suaves)
  - Diferentes idiomas (Unicode, RTL, ligaduras)
- **Solução tradicional**: Bitmap fonts (imagem com todos caracteres)
- **Solução moderna**: FreeType (rasterização dinâmica)
- **Usado por**: Unreal Engine, Unity, Firefox, Chrome, Photoshop, Blender



Camada 3: Outras Bibliotecas Úteis

- Eigen – Álgebra Linear Genérica
- Assimp – Carregamento de Modelos 3D
- FastNoise – Ruído Procedural



Camada 4: Shaders e Compiladores de Shaders

- Problema: GPU não entende C/Python

```
// Isso NÃO funciona na GPU:
for (int i = 0; i < vertexCount; i++) {
    output[i] = transform(vertices[i]);  // GPU não executa loops assim!
}

// GPU precisa de um modelo SIMD (Single Instruction, Multiple Data)
// O mesmo programa executa em MILHARES de vértices/pixels simultaneamente
```

Arquitetura da GPU vs CPU:

Característica	CPU	GPU
Núcleos	Poucos (8-16) potentes	Milhares (4096+) simples
Execução	MIMD (múltiplas instruções)	SIMD (mesma instrução em massa)
Cache	Grande (L1, L2, L3)	Pequena (textura apenas)
Memória	Cache-coerente	Massivamente paralela
Controle de fluxo	Excelente (branches complexos)	Pobre (divergência reduz performance)



Camada 4: Shaders

- Pergunta: O que exatamente é um shader?
  - Shader é um pequeno programa que roda na GPU, especializado em processar gráficos;
  - Shader é um programa executado pela GPU que define COMO cada vértice ou pixel será processado no pipeline gráfico

Característica	Descrição
Executa na GPU	Não roda na CPU
Altamente paralelo	Milhares de instâncias rodam juntas
Especializado	Cada tipo faz uma etapa específica
Sem efeitos colaterais	Não acessa memória global arbitrária
Limitado	Sem chamadas de sistema, sem alocação dinâmica



Camada 4: Shaders – Histórico

- **Origem histórica**: "Shade" = sombreamento (em inglês)
- Os primeiros shaders (1980s) só faziam **sombreamento**:
- Calcular como a luz reflete na superfície
- Definir a cor final do pixel
- **Hoje** shaders fazem MUITO mais:
  - Distorcer tela (efeito de água)
  - Desenhar padrões complexos (fractais)
  - Processamento de imagem (filtros)
  - Simulações físicas paralelas
  - O nome "shader" ficou por tradição.



Camada 4: Shaders – Funções

Tipos de Shader e suas Funções

Tipo	Executa em	Função	Exemplo uso
Vertex	Cada vértice	Posicionar pontos 3D na tela	Animação de personagem
Fragment	Cada pixel	Definir cor final	Textura, luz, efeitos
Geometry	Cada primitiva	Criar/destruir vértices	Grama, explosões
Tessellation	Patch de vértices	Subdividir superfície	Terreno detalhado
Compute	Qualquer dado	Computação geral	Física, partículas



Camada 4: GLSL

- GLSL – OpenGL Shading Language
- **GLSL** é uma linguagem de programação **especializada**, com sintaxe similar ao C, projetada exclusivamente para escrever shaders que rodam na GPU

Características Fundamentais

Característica	Descrição
Tipo	Domain-Specific Language (DSL)
Sintaxe	Similar a C, mas com restrições
Execução	Altamente paralela na GPU
Compilação	Em tempo de execução (ou offline para SPIR-V)
Versões	1.10 (2004) → 4.60 (2018), ES para mobile

O que NÃO existe em GLSL?

Diferente de C/C++, GLSL não tem:

- Ponteiros
- Alocação dinâmica de memória (`malloc` / `new`)
- Recursão
- Chamadas de sistema
- I/O (arquivos, console)
- `goto` ou `switch` (em versões antigas) 8



Camada 5: Motores Gráficos

- Mais alto nível de abstração que um desenvolvedor de aplicações em computação gráfica pode utilizar.
- Um motor gráfico é um framework completo que gerencia todos os aspectos da renderização 3D/2D, incluindo:
  - Cena 3D (hierarquia de objetos)
  - Materiais e shaders
  - Iluminação e sombras
  - Câmeras
  - Animação
  - Partículas
  - Pós-processamento
  - Otimização de renderização



Camada 5: Motores Gráficos

- Ursina Engine - Motor Gráfico Python Puro
- Por que Ursina?
- Ursina é construído sobre Panda3D (motor C++ com bindings Python), mas oferece uma API extremamente simples.
- Vantagens:
- Código mínimo (10 linhas = jogo 3D)
- Entrada/saída simplificada
- Ideal para ensino e prototipagem



Camada 5: Motores Gráficos

- Panda3D - Motor Profissional com Python
- Por que Panda3D?
- Panda3D é um motor C++ com bindings Python oficiais. Usado por grandes estúdios (Disney, NASA).
- Características:
- Pipeline de renderização completo
- Suporte a shaders personalizados
- Otimizado para performance
- Usado em produção



Camada 5: Motores Gráficos

- Three.js + Python (Via PyScript ou Flask)
- Por que Three.js?
- Three.js é o motor Web mais popular para gráficos 3D no navegador.
- Com Python, você pode:
- Servir páginas com Three.js via Flask
- Usar PyScript para rodar Python no navegador

